

CAS4401 Assignment 2 Report

김환욱

1. Source Code High-Level Description

The implementation simulates a decentralized learning environment where multiple clients collaborate to train a global CNN model while minimizing communication overhead. The code is structured into three main components:

a. Local Client-Side Training (train)

Each client receives the latest global model weights and performs local Stochastic Gradient Descent (SGD) on its private dataset. Instead of returning raw weights, the function calculates '**pseudo-gradients**' (the difference between the global model and the locally updated model divided by the learning rate). This ensures that the server can update the global model in a manner consistent with standard optimizer steps.

b. Gradient Compression and Error Feedback (topk_pruning_error_feedback)

To reduce bandwidth usage, we implement **Top-k Sparsification**.

- **Sparsification:** Only the top k% of gradient elements (by absolute magnitude) are transmitted.
- **Error Feedback (EF):** This is the core logic for robustness. Pruned gradients (those not in the top-k) are stored in a local **residual buffer**. In the next round, these residues are added to the new gradients before the next pruning step. This mechanism ensures that "small but important" information is not permanently lost but only delayed.

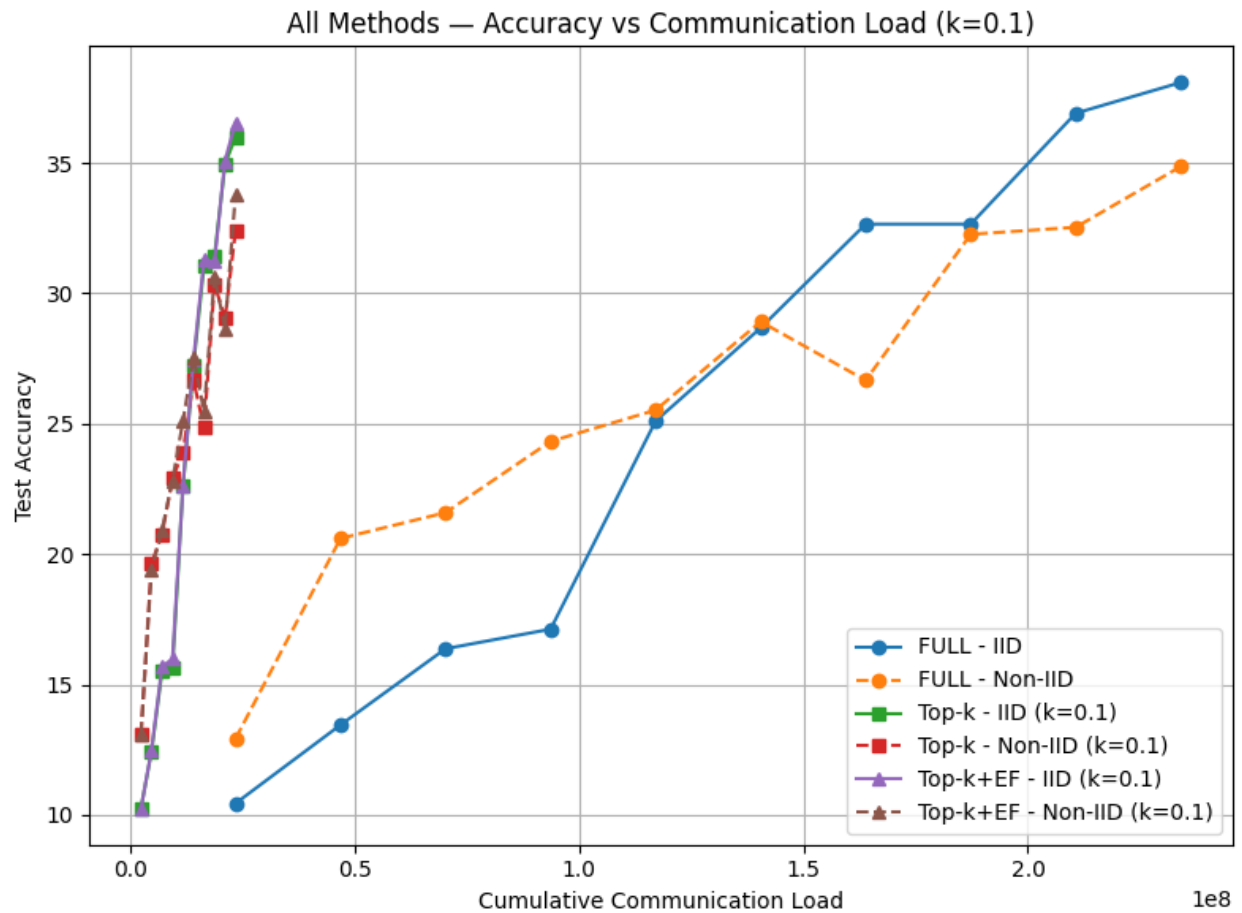
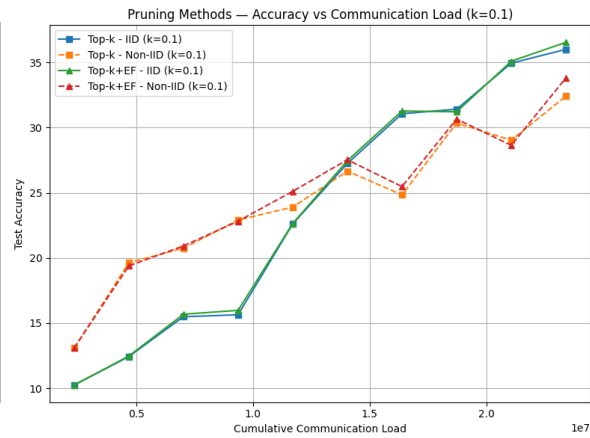
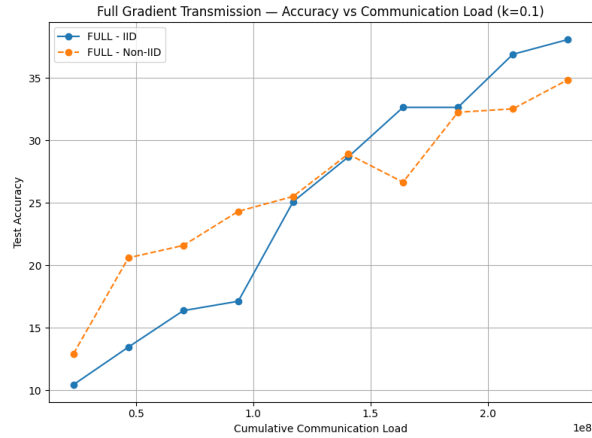
c. Federated Learning Loops (Sync vs. Async)

- **Synchronous FL (sync_fl):** Implements the FedAvg protocol where the server waits for all clients to finish. It uses $\max(\text{CLIENT_DELAYS})$ to simulate the total time per round, highlighting the **Straggler Problem**.
- **Asynchronous FL (async_fl):** Implements a real-time update scheme. As soon as a client finishes, the server updates the global model using a **Staleness-aware Alpha** ($\alpha_{\text{star}} = \alpha / (\text{staleness} + 1)$) to mitigate the negative impact of outdated gradients.

2. Task 1: Pruning & Error Feedback Analysis

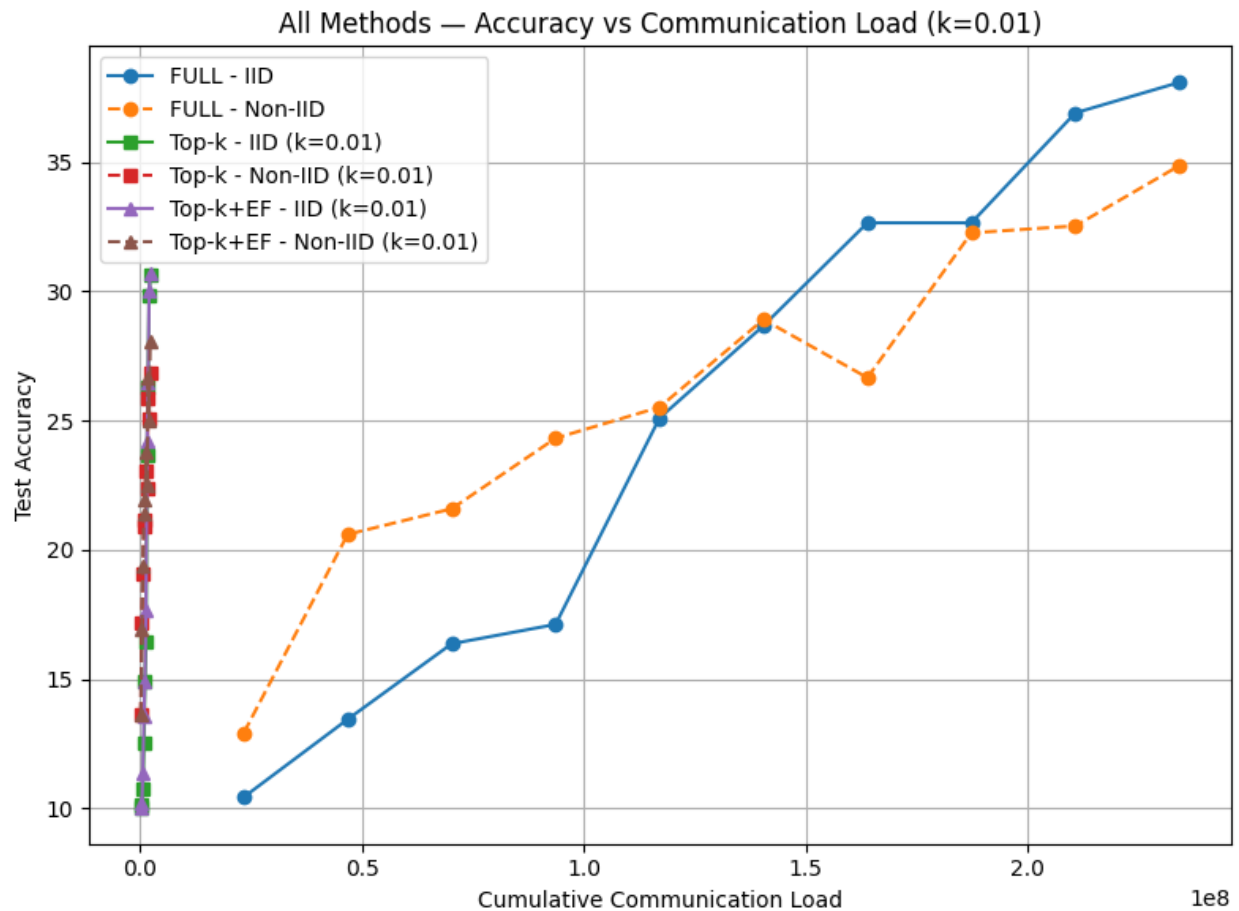
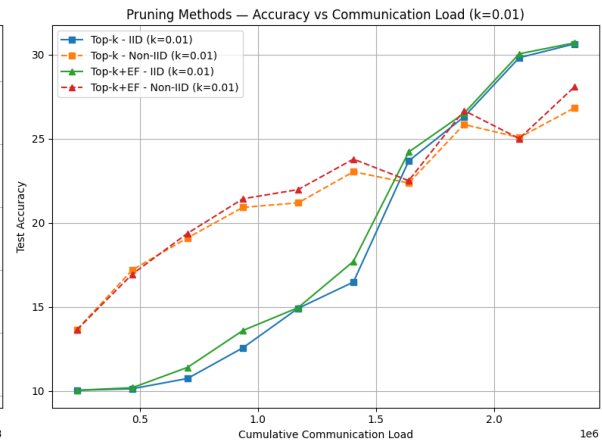
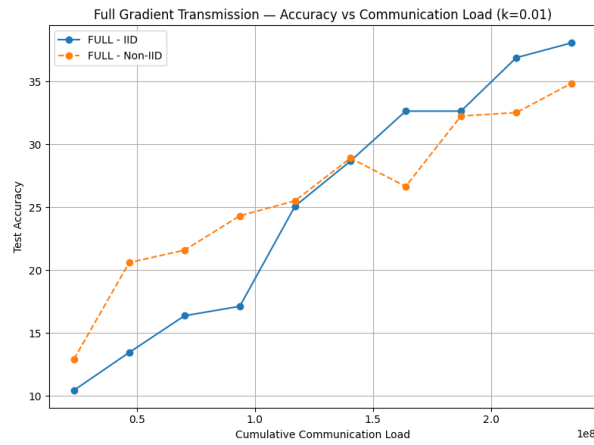
Comparison for $k_{\text{ratio}} = 0.1$

At a higher retention rate ($k = 0.1$), the global model receives a relatively strong signal even without error correction.



Comparison for $k_{\text{ratio}} = 0.01$

At extreme sparsity ($k = 0.01$), the role of Error Feedback becomes critical.

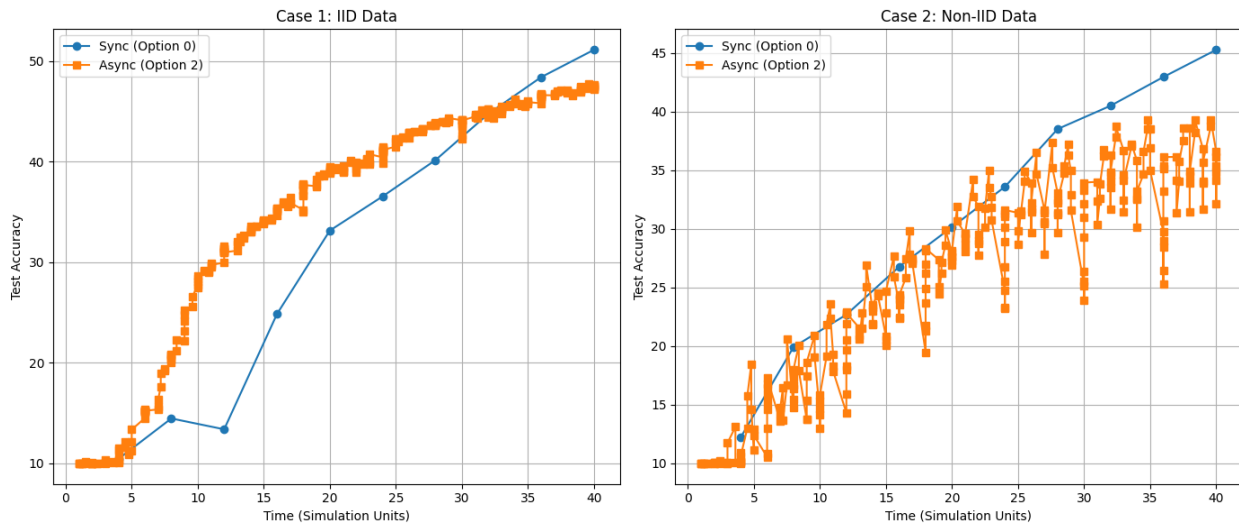


Answers to Task 1 Questions

- **Best Method:** Top-k+EF reaches the highest accuracy given the same communication load. While Full Transmission is accurate, its high "cost per update" means it performs fewer updates for the same total data spent. Top-k+EF provides the best "accuracy per bit" by correcting pruning errors over time.
- **Non-IID Impact:** Accuracy degrades in Non-IID cases because client gradients are biased toward local classes. Pruning biased gradients risks discarding "minority class" information that is essential for the global model's generalization.
- **EF in Later Rounds:** In later stages, gradients become smaller. Plain Top-k often discards these small values, leading to a plateau. Top-k+EF accumulates these small values in the residual buffer until they are large enough to be sent, allowing for fine-grained fine-tuning.
- **K_ratio Effect:** The gap is smaller at $k=0.1$ because the "discarded" info is only 90%. At $k=0.01$, we discard 99% of the signal; without EF to "remember" that 99%, the model loses its direction entirely.

3. Task 2: Synchronous vs. Asynchronous Analysis

This section evaluates how the system handles heterogeneous client delays (stragglers).



Answers to Task 2 Questions

- **Convergence & Stability:** **Synchronous** settings are highly stable and converge smoothly but are slow due to the "Straggler Problem" (waiting for the slowest client). **Asynchronous** settings converge faster in terms of simulation time but are prone to fluctuations due to "Staleness" (updates based on old model versions).

- **Shift from IID to Non-IID:** When shifting to Non-IID, the **Sync** strategy handles data heterogeneity better. This is because Sync averages all biased updates simultaneously, which helps cancel out individual client biases. In **Async**, a single biased update can immediately push the global model in a wrong direction before other clients can provide counter-balance.

4. Conclusion and Critical Insights

The experimental results highlight a fundamental trade-off:

- **Communication vs. Accuracy:** High compression ($k=0.01$) is only possible with **Error Feedback**. EF is the "memory" of the system that ensures mathematical consistency over time.
- **Time vs. Stability:** **Asynchronous FL** is ideal for real-world scenarios with varying device speeds, but it requires a **Staleness-aware learning rate** to maintain stability, especially when data is Non-IID.

In conclusion, for a robust distributed system, **Top-k+EF** should be paired with **Synchronous** updates for maximum accuracy, or with **Asynchronous** updates (and strict staleness control) for maximum time efficiency.